Curtin University

SPAT 5008 - Applied GIS

Lab 6: Animal Movement – Snakes in Cape York

Written by Dr. Todd Robinson (T.Robinson@curtin.edu.au)

Learning Outcomes

After completing this lab, you should be able to:

- Understand the main difference between Lagrangian and Eulerian movement data.
- Extract time and projection information and understand both the POSIX and PROJ4 formats.
- Compute basic summary statistics from movement data and identify outliers for potential deletion.
- Compute the home range using convex hull and kernel density estimation.
- Compute dynamic Brownian bridge movement models that incorporate TIME and changes in the behaviour (foraging, sleeping, migrating) in the utilisation distribution model.

Your tasks:

- Download the dataset from the Blackboard.
- Complete the tutorial.
- NO answers required this week.

Dataset Source:
Data on the green python (*Morelia viridis*) in Cape York, was downloaded from https://www.movebank.org/cms/webapp?gwt_fragment=page=studies

INTRODUCTION

Animal movement analysis for tracking animals, seeking covariates with landscapes, and determining their range has received considerable attention over the last few decades. Not surprisingly, with the improvement in ways to track animals, such as smaller and more robust GPS collars, a greater number of studies on a greater variety of animals has followed. In my opinion there is little doubt that this will continue to exponentiate, and the spatial experts will be right there helping ecologists explore the movement processes of the Great White Shark, migratory birds, and Galapagos tortoises. What a remarkably interesting career that would be!

Movement data are just data that tracks the change of an object through time. To get you thinking, what about the movement patterns of burglars? Do they stick to the same patch or randomly burgle? Are they territorial? Some interesting work has been done in this area using the same principles as applied to animals, where the patterns of offences are analysed by different levels of offenders – from the beginner to the advanced. See Johnson (2014) for a description of the "offender as a forager". More topical – how about tracking the Coronavirus? Contact tracing produces a similar dataset to that which we will use in this lab – and LOTS of it! The question to ask is, where did Person X spend most of his/her time whilst potentially shedding the virus. Those people may need to isolate.

The obvious difference to anything we have studied so far in this unit is that we are no longer considering individual points as individual events. Instead, we now have a point, which is connected by another point, which is connected by more points that are all related to one individual. These points are usually generated at regular intervals, such as every 30 minutes, once a day, once a week etc. Furthermore, often more than one individual is tagged, but rarely will you have data at the population level, so it is important to temper analyses

to the sample you have access to. Analysis tools and techniques need to adapt to these differences – one individual animal or a sample of individuals? Whilst analysing multiple individuals at one time is possible, it is more common to analyse each with the same tool several times. Here, we will just look at one individual to give you the skills to repeat it and if desired find patterns you are able to generalise.

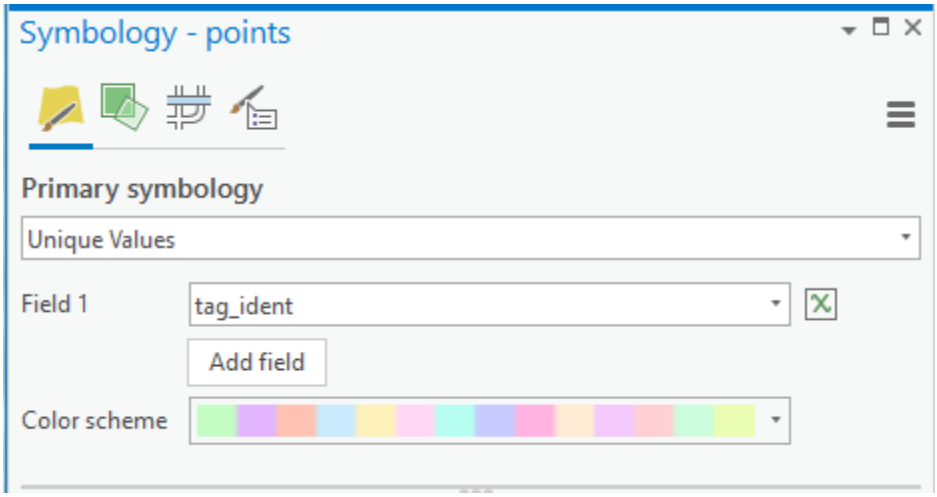
Animal tracking data is usually collected by one or both of two major methods – the Lagrangian and the Eulerian methods. The distinguishing factor between them is that under the Lagrangian method individuals are tagged using a radio tracker, GPS or special bands, which ping a location every so often. Under the Eulerian methods, many locations are continuously surveyed by camera traps, which are triggered if an animal walks past them; they are therefore limited to the observation arena they are set up for. Their analyses are different and I'll leave it to you to research more about that. Here we will focus only on data collected with the Lagrangian method. If you ever have camera data at your disposal, have a look at camtrapR (Niedball et al., 2016).

EXERCISE 6.1 – GETTING STARTED

Start ArcGIS PRO and create a new map by clicking on the Map template icon. You will need to name your project and set a working folder location. Enter lab_06_pythonmovement for the name and then choose an output folder (e.g. Lab6_Data).

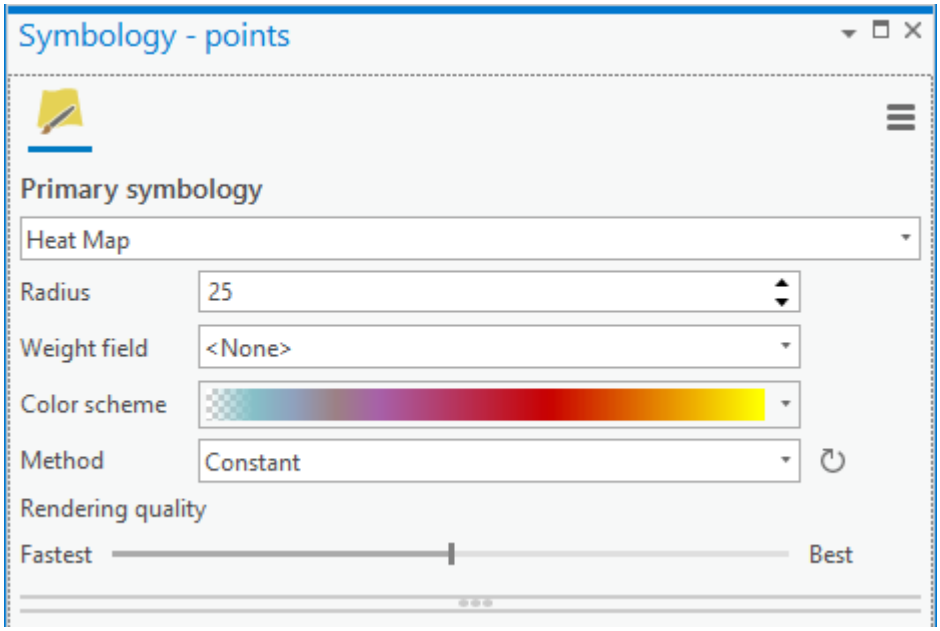
Remember to click on View>Catalog Pane to see your data sources and Insert> Add Folder if the one you want is not there. Browse to the folder where you placed the tutorial data off the Blackboard and connect to the EX61 folder. Add in the point and line data from this folder.

Right click on the point layer and symbolise Unique Values according to tag_ident:

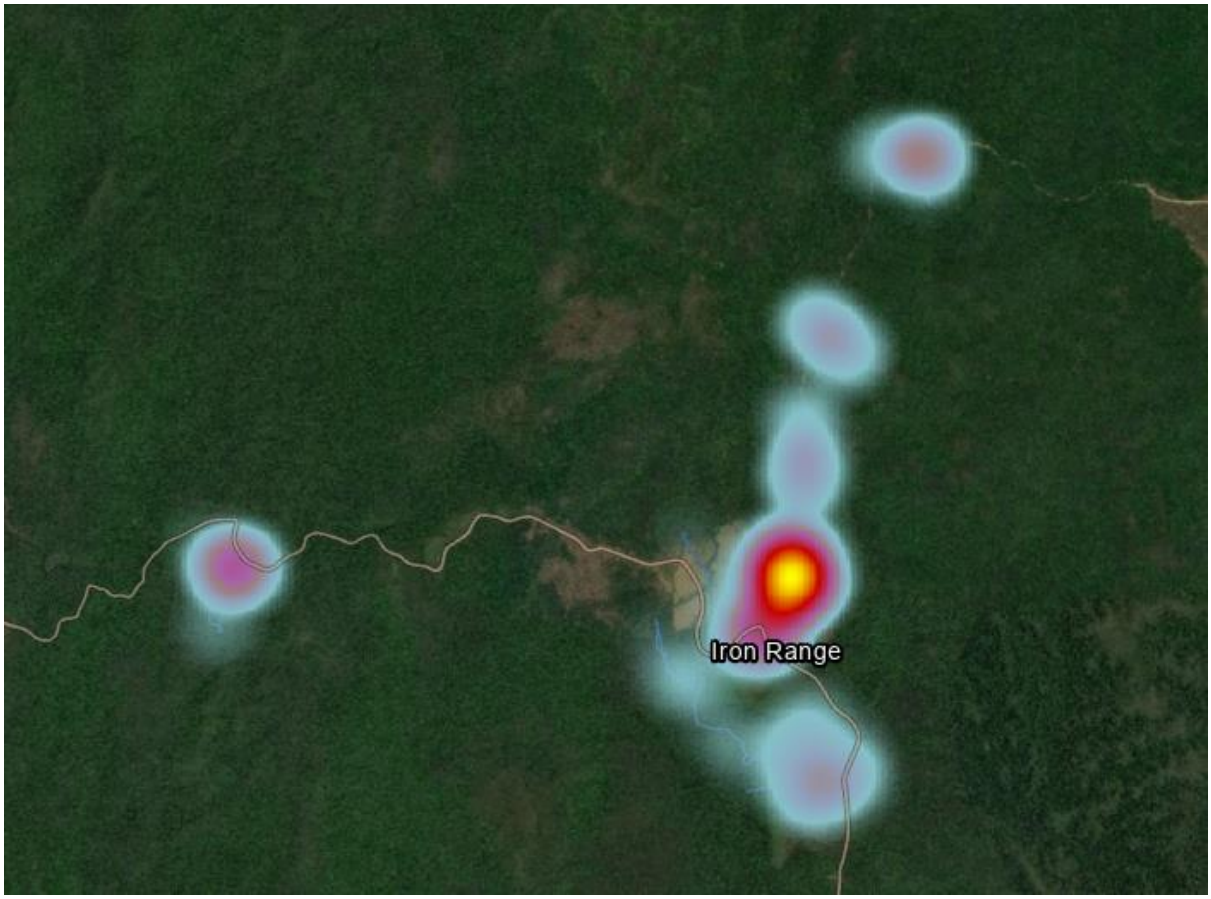


As the basemap, choose Imagery Hybrid under Map and then just explore the habitat of these pythons. There's clearly a couple of them that have ventured outside of their tree canopies and into the open spaces but in general, they have well-defined smallish ranges, relative to e.g. a migratory bird. Not being the author of the data, I currently assume that the association with dirt tracks to be a sampling artefact ("convenience sampling") rather than a habitat preference.

Choose Symbology again and this time choose Heat Map. Leave all other values as default.



Zoom to layer and you note that most of the “heat” is around Iron Range. One of the individuals found in that cluster is tagged number “2”. I’m going to restrict our analyses to this number 2...



EXERCISE 6.2 – INSTALL R & R-STUDIO

If you are working in the Spatial labs, on Spatial computers, you can skip to the next section. Otherwise, navigate to https://cran.rstudio.com/ to download R. Under Download and Install R, choose the link for your OS (e.g. Download R for Windows). At this point we just want R, so choose “base” at the next screen. Then download the latest version of R (4.x.x for Windows). Run the executable and accept all defaults.

Now, navigate to https://rstudio.com/products/rstudio/ to download R-Studio. Choose the download RSTUDIO DESKTOP button from the Open Source Edition. Then click this

2: Install RStudio



Size: 212.77 MB | SHA-256: A8325AD5 | Version: 2023.06.1+524 | Released: 2023-07-07

Now, install R-Studio by accepting all and clicking next and Finish. That's it. Now Run R-Studio.

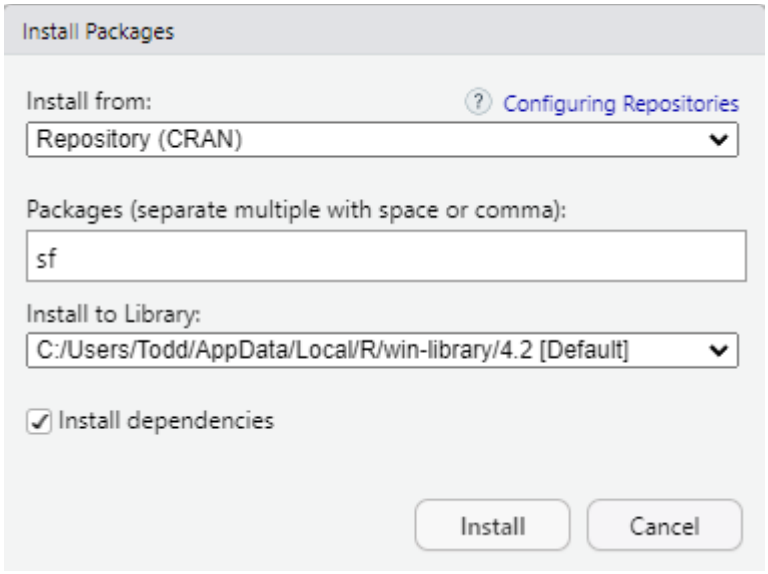
EXERCISE 6.3 – READING IN DATA

The data we received was in both shapefile and csv format. I have put the csv format data in the EX63 folder.

Open the CSV labelled Green_python2 in Excel just to inspect it. You will notice an event id, a timestamp, lat and long data, sensor type, tag id, an individual id, and the name of the study.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	event_id	visible	timestam	long	lat	sensor_ty	tag_local_	individ	study_name				
2	1.31E+10	TRUE	17:00.0	143.2838	-12.7429	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				
3	1.31E+10	TRUE	07:00.0	143.2838	-12.7429	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				
4	1.31E+10	TRUE	00:00.0	143.2837	-12.743	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				
5	1.31E+10	TRUE	10:00.0	143.2835	-12.7429	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				
6	1.31E+10	TRUE	18:00.0	143.2836	-12.7428	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				
7	1.31E+10	TRUE	48:00.0	143.2835	-12.7426	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				
8	1.31E+10	TRUE	20:00.0	143.2836	-12.7423	radio-tran	#10061466	2	Green python Morelia viridis, Cape York Australia				

Go back to your R-Studio session now. The first thing we need to do is install a package. The easiest way to do this is click Packages, then install and then type sf (for simple features).



Click Install. After it runs through its installation procedure, find it in the list of packages and make sure it is ticked on. Note: Your list of packages may be different to mine.

☐	scales	Scale Functions for Visualization	1.2.1
☐	servr	A Simple HTTP Server to Serve Static Files or Dynamic Documents	0.25
☐	sessioninfo	R Session Information	1.2.2
☑	sf	Simple Features for R	1.0-14
☐	sfheaders	Converts Between R Objects and Simple Feature Objects	0.4.0
☐	sftime	Classes and Methods for Simple Feature Objects that Have a Time Column	0.2-0
☐	shiny	Web Application Framework for R	1.7.4

You should generally set up a working directory for any project, especially if you wish to export some of the outputs.

Please adapt the path and change the wd using setwd as shown in the following example:

```
wd<-
setwd("C:/Users/Todd/OneDrive/Teaching/AppliedGIS/
Lab6/EX63")
```

If this does not show up in your values correctly, run the line again until it does. For some reason, it often takes two goes.

```
values
  wd                    "C:/Users/Todd/OneDrive/Teaching/AppliedGIS/Lab6/EX63"
```

Now, read in the file. It should say pythons, 251 obs of 9 variables.

```
python<-read.csv("Green_python2.csv", as.is=TRUE)
```

How does it look? Have a look at the first few rows with the head command.

```
head(python)
```

```
> head(pythons)
     event.id visible             timestamp location.long location.lat       sensor.type
1 13077481698    true 2003-01-09 12:00:00.000      143.2433    -12.74131 radio-transmitter
2 13077481699    true 2003-01-15 01:50:00.000      143.2435    -12.74127 radio-transmitter
3 13077481700    true 2003-01-15 22:49:00.000      143.2437    -12.74106 radio-transmitter
4 13077481701    true 2003-01-22 07:50:00.000      143.2432    -12.74517 radio-transmitter
5 13077481702    true 2003-01-24 02:43:00.000      143.2431    -12.74537 radio-transmitter
6 13077481703    true 2003-01-25 01:26:00.000      143.2432    -12.74551 radio-transmitter
```

Currently our timestamp is a character string that cannot yet be used to calculate movement. We need to convert it into a machine-readable time class known as POSIX – standing for Portable Operating System Interface format. This just requires unscrambling the timestamp into its major components: year, month, day and so on.

To learn more about this Date-Time class (or any command you want to try) use the ?command syntax:

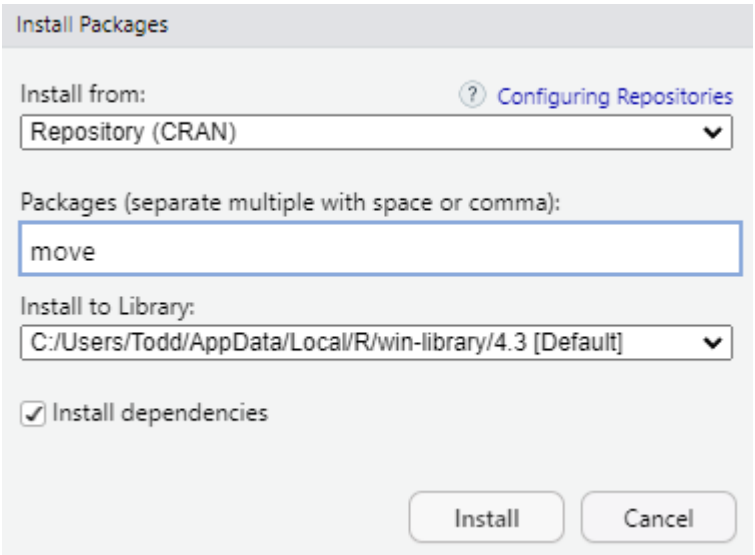
```
?POSIXct
```

This will load the help description for POSIXct. Have a quick read of that, especially look at the examples of code at the bottom. Frequently, you can plug into those quite easily. Now make a new variable called pythontime in this format and a universal time zone.

```
pythontime<-as.POSIXct(python$timestamp,
```

```
format="%Y-%m-%d %H:%M:%S", tz="UTC")
```

Now we will create a move object defined by the move package. Please use Packages and install “move”.



Once installed check that it is ticked in the package list. Copy ALL of the code below and paste into R-Studio.

☐	memoise	'Memoisation' of Functions	2.0.1
☐	mime	Map Filenames to MIME Types	0.12
☑	move	Visualizing and Analyzing Animal Track Data	4.2.4
☐	openssl	Toolkit for Encryption, Signatures and Certificates Based on OpenSSL	2.1.0
☐	pillar	Coloured Formatting for Columns	1.9.0

```
mypython<-move(

#set coordinates

x=python$long, y=python$lat,

#provide my time format in POSIX

time = pythontime,

#set the projection - this is PROJ4 format. See
```

```
below for more details.

proj = "+proj=longlat +ellps=GRS80 +no_defs",

#set the id

animal=as.numeric(python$individ),

#add the rest of the data

data = python)
```

Now we will project our data into metres. If you do not know which UTM zone your projection is in then take your longitude, add 180 and divide by 6 and round to the next integer (ceiling). This will give you 53.8. Hence, our data is in UTM Zone 54. To make a projection, first you need to define the current coordinate system. We have done that above with the line +proj=longlat + ellps = GRS80 + no_defs. That means we can now project it to something else. Both defining a projection and doing the projection requires some basic knowledge of the PROJ4 syntax. This is my go to for that https://mgimond.github.io/Spatial/coordinate-systems-in-r.html.

Essentially, you need to specify that it is a UTM projection, the zone, south or north, the datum, and the units. No_defs means no defaults, i.e., nothing other than what we prescribe below.

```
mypython_prj<-spTransform(mypython, "+proj=utm

+zone=54 +south +datum=WGS84 +units=m +no_defs")
```

Exercise 6.3.1 – Summary Statistics and Cleaning

In this next stage of the analysis, we are going to run our data through some simple statistics: distance and speed. The idea of this is twofold. First, you get to make inferences about your animal, such as whether it likes to hang around

in the same spot (like a snake), or make large distance runs (like a salmon). Second, you can screen your data for unusual observations, i.e., outliers. Try out the following commands. I think you will find that our snake doesn't really travel great distances, and never at great speed.

```
#distance, in m, between points.

distance<-distance(mypython_prj)

hist(distance)

#speed is in m per second.

speed<-speed(mypython_prj)

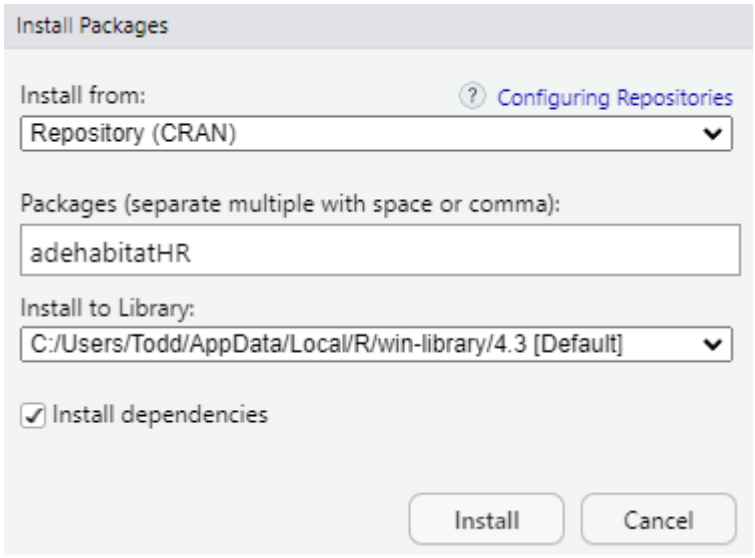
hist(speed)
```

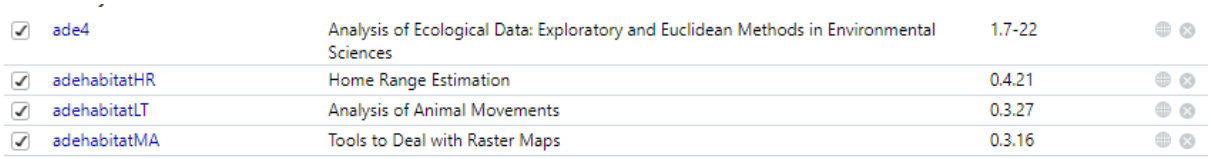
After looking at the data in this way, I do not think there are any obvious errors. We shall proceed as is. Another cleaning routine is to ensure your animal is not found in any locations it cannot be – e.g. a pig in deep ocean, a fish on land. We essentially did that when we viewed the data in ArcGIS.

EXERCISE 6.4 – THE HOME RANGE

The basic definition of an animal's home range is the spatial extent of its movements that are required in its normal activities to maintain its needs, which include finding a meal, a partner, sleeping, residing and caring for young. It does not include migration and should seek to eliminate random "walks" – unusual and brief excursions that rarely, if ever, repeat.

We will begin with a minimum convex polygon (MCP) method using the adehabitatHR package. Install it and make sure it is checked. By ticking this on, it will tick on a few other dependencies. That's fine.





The MCP method encloses the outer locations to make a home range polygon. Usually, around 90% (it's up to the user) of the points are retained to remove random walks.

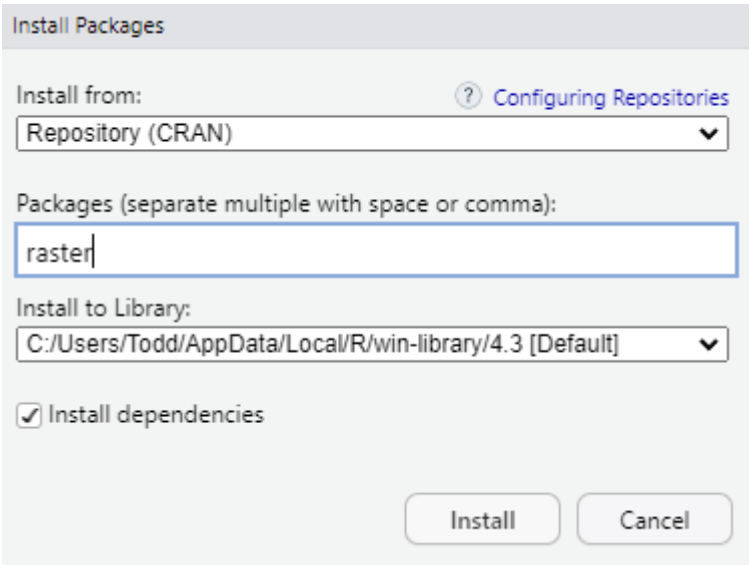
First, we convert to a SpatialPoints class and then we use the mcp command:

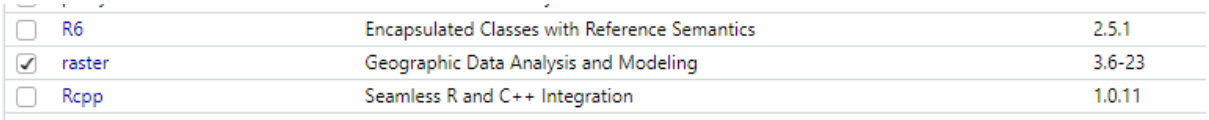
```
mypython_pts<-as(mypython_prj, "SpatialPoints")

mypython_pts.mcp<-mcp(mypython_pts, percent = 90,
unin = "m", unout = "ha")

mypython_pts.mcp
```

So, it would appear, our snake has a home range of about 6 ha, according to the mcp method. It would certainly be interesting now, to see how that compares to the other snakes that have been monitored. However, for now, let's view the home range on a Sentinel image. Install and check the package named "raster".






```
#Change this to suit your path!
wd<-
setwd("C:/Users/Todd/OneDrive/Teaching/AppliedGIS/
Lab6/EX64")
wd<-
setwd("C:/Users/Todd/OneDrive/Teaching/AppliedGIS/
Lab6/EX64")
```

```
TCC<-stack('TCC.tif')

plotRGB(TCC, 3, 2, 1, stretch = "lin")

points(mypython_prj, pch = 20, col = "yellow")

plot(mypython_pts.mcp, add = TRUE, border = "red",
lwd = 3)
```

Our final MCP appears below. This is the approximate home range of python number 2. The points to the west are a random foray and therefore not part of its home range using this method.



Let's save that as a shapefile so we can use it in ArcGIS. We can use the sf package. No need to reinstall.

```
#We first convert our spatialpolygonsdataframe to
an sf object.

mcp<-st_as_sf(mypython_pts.mcp)

st_write(mcp, dsn = "mcp_py2.shp", layer =
"mcp_py2.shp", driver = "ESRI Shapefile", append =
FALSE)
```



One of the drawbacks of the MCP approach is that it is sensitive to sample size. In addition, the 90% threshold is just a rule of thumb to deal with outliers, but it may not exclude them all, or it may exclude too many. For example, one extreme point left in the solution that is still in the top 90% could inflate the convex polygon by many ha, especially if you are dealing with an animal that moves quickly. Therefore, another technique is to model the utilization distribution. In other words, the more an animal is found in the same spot, or close to it, the more it is likely to be suitable habitat and part of its home range. This concept is the same as the very quick heat maps we produced in ArcGIS earlier. It uses a moving kernel to estimate the utilisation of space. Here we will use the default “href” for the smoothing parameter, but you can change it if you like. The higher you make it, the more smoothing will occur and vice versa.

```
mypython_pts.kernel<-kernelUD(mypython_pts, h =

"href")

mypython_pts.kernelarea<-
kernel.area(mypython_pts.kernel, unin = "m", unout
= "ha")

mypython_pts.kernelarea
```

The last function will output the different areas for different levels of utilisation. I've chosen to output a kernel of 65% and of 90%.

```
plotRGB(TCC, 3, 2, 1, stretch = "lin")

points(mypython_prj, pch = 20, col = "yellow")

plot(getverticeshr(mypython_pts.kernel, percent =
65), lwd = 3, border = "red", lwd = 3, add = TRUE)

plot(getverticeshr(mypython_pts.kernel, percent =
90), lwd = 3, border = "grey", lwd = 3, add =
TRUE)
```

The output of the two scenarios is found below.



For comparison sake, let's output the second one as a shapefile where percent = 90.

```
kernel90<-getverticeshr(mypython_pts.kernel,
percent = 90)

kernel90<-st_as_sf(kernel90)

st_write(kernel90, dsn = "kern90_py2.shp", layer =
" kern90_py2.shp ", driver = "ESRI Shapefile",
```

```
  append = FALSE)
```

Now bring that into ArcGIS and compare to the MCP.



Neither of the two approaches examined so far are really taking advantage of the TIME factor, which makes this dataset so unique and powerful. Brownian bridge movement models do incorporate the temporal (time) structure of the data by integrating the order of locations and the amount of time between them. In effect, they model the utilisation distribution based on the movement path, rather than just how tightly clustered the recorded points are, as with the previous two models. For example, a foray might be identified as locations that were not stopped at for long, followed by more locations of the same, and none of them are frequently, if ever, revisited. A good paper on this is one by Kranstauber et al. (2012) who also go into the mathematics required to do it. They describe how the movement path between A and B are modelled by applying a conditional random walk, which considers how much variance can be expected when travelling in a straight line from A to B – the variance of the Brownian motion.

They extend the concept to make it more "dynamic" using a sliding window approach where the variance of the Brownian motion is computed iteratively to account for an animal moving in different ways such as when foraging, travelling between sites and periods of rest. For example, a migratory bird will move over a small range when breeding, but over a long range when migrating. Thus, one value for the Brownian motion parameter is insufficient at

estimating the whole trajectory, underestimating some paths, and overestimating others. Underestimation leads to utilisation distributions that are too narrow and overestimation to wider utilisation distribution areas.

The parameters required to compute a dynamic Brownian bridge are:

Object - a move object, hence mypython_prj can be used.

Raster - is the resolution of the output probability surface. 10 m corresponds well to the Sentinel imagery, so I have chosen that, but it need not be.

Location error – Potential amount of error from the sender/receiver system. I have guessed at 19 m. A bit more than a pixel.

Margin - The margin at the start and end of the moving window to avoid edge effects.

Window size – The size of the moving window along the track. The larger the window, the more stable the estimates are of the Brownian motion variance, but poorer the ability to capture frequent behaviour changes. The window size must be odd.

```
myDBBM<-

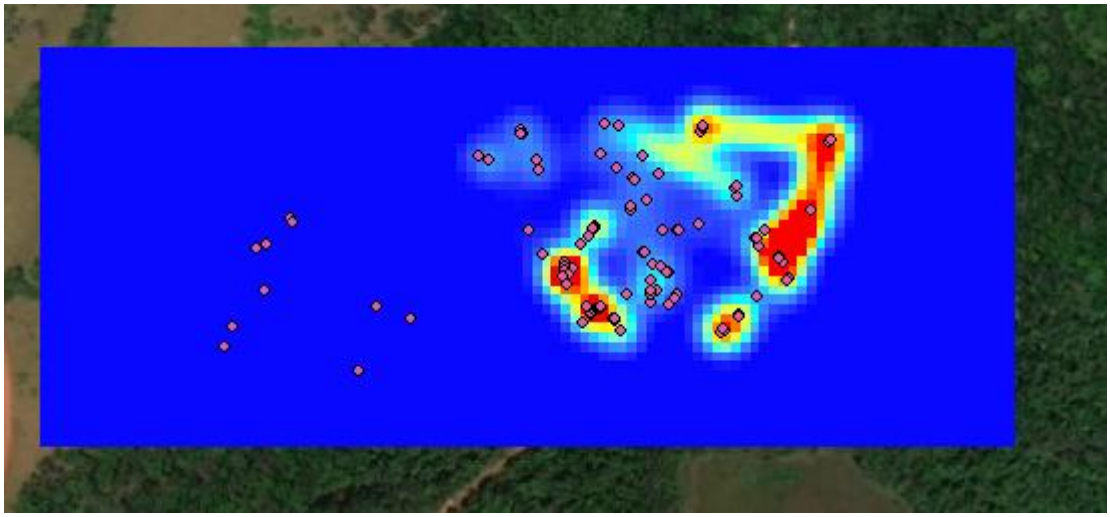
brownian.bridge.dyn(mypython_prj, raster = 10,
location.error = 19, margin = 5, window.size = 21)

plotRGB(TCC, 3, 2, 1, stretch = "lin")

plot(myDBBM, add = TRUE)
```

You may like to write it out as a tif and view in ArcGIS.

```
writeRaster(myDBBM,
filename="BBmodel.tif", options="INTERLEAVE=BAND",
overwrite=TRUE)
```



Now, to export this as a polygon we have a little more work to do, since it's currently a raster. Firstly, we need to convert the output to a probability, which we do using getVolumeUD. Then we need to convert that to contours, the contours to polygons, subset the 90% polygon and then export it.

```
UDmypy<-getVolumeUD(myDBBM)

plot(UDmypy)

UDmypy_conts<-rasterToContour(UDmypy, nlevels =
20)

mycontours<-st_as_sf(UDmypy_conts)

my_contours_poly<-st_polygonize(mycontours)

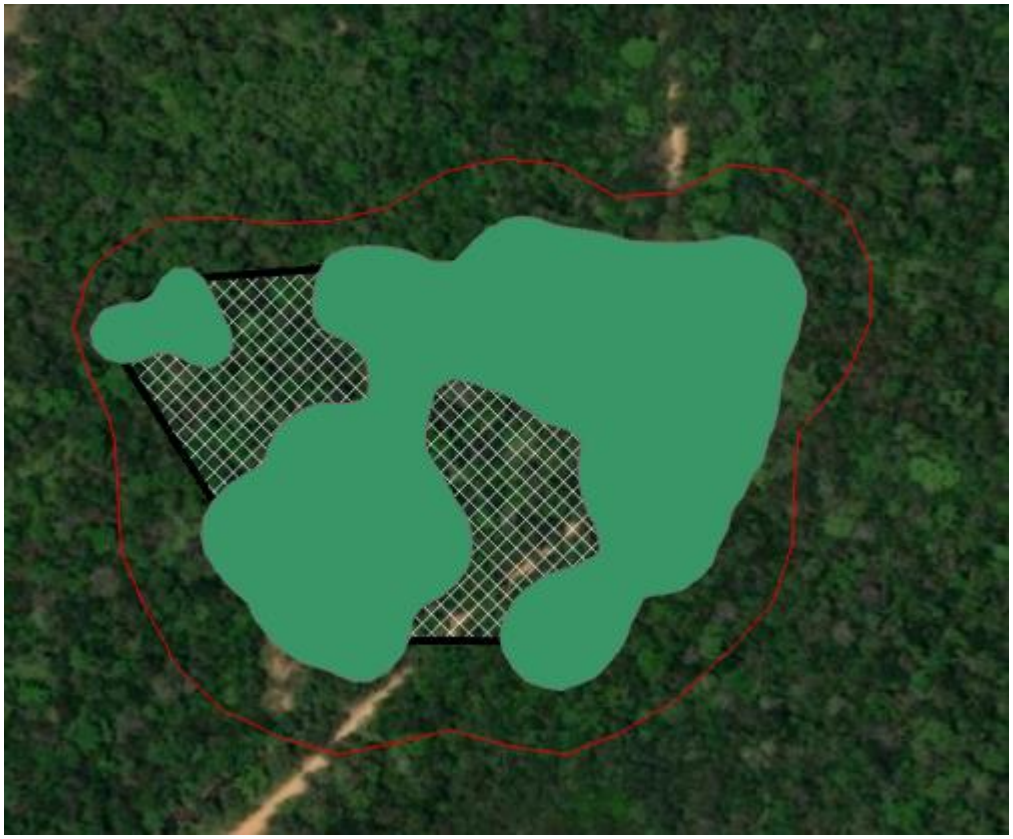
my_contours_subset<-subset(my_contours_poly, level
== 0.9)

my_contours_dissolved<-
st_union(my_contours_subset)

st_write(my_contours_dissolved, dsn =
"dbbm90_py2C.shp", layer = "dbbm90_py2C.shp ",
```

```
driver = "ESRI Shapefile", append = FALSE,
layer_options = "SHPT=POLYGON")
```

Once again, bring that file into view in ArcGIS. I think you'd agree that this technique produces a more nuanced utilisation distribution.



So, there you have it. The points to the west are quite likely to be a result of foray and not where it spends a great deal of its time. The previous two methods also identified that but were less exclusive of the points in the centre and northwest, which were clustered but were not utilised for any great length of time. So, if we were looking to find Python2, say to refit a broken receiver, there appears to me that it has a few favourite trees in its relatively small range. Of course, there are obvious assumptions in the parameters chosen and it would be prudent to test other variations to determine their sensitivity in the solution. But, hopefully you get the idea!

References

Johnson, S. (2014). How do offenders choose where to offend? Perspectives from animal foraging. Legal and Criminological Psychology, 19(2), 193-210. doi: 10.1111/lcrp.12061

Kranstauber, B., Kays, R., LaPoint, S., Wikelski, M., & Safi, K. (2012). A dynamic Brownian bridge movement model to estimate utilization distributions for heterogeneous animal movement. Journal of Animal Ecology, 81(4), 738-746. doi: 10.1111/j.1365-2656.2012.01955.x

Niedballa, J., Sollmann, R., Courtiol, A., & Wilting, A. (2016). camtrapR: an R package for efficient camera trap data management. Methods In Ecology and Evolution, 7(12), 1457-1462. doi: 10.1111/2041-210x.12600

End of Session.